

Evolutionary Weightless Neural Networks for Network Intrusion Detection

Anonymous

Anonymous Institution

Abstract. Weightless Neural Networks (WNNs) based on Random Access Memory (RAM) neurons offer constant-time inference with sub-microsecond lookup, making them attractive for edge-deployed intrusion detection systems. However, their classification performance depends on *connectivity*, which input bits each neuron observes, a problem traditionally solved by random initialization. We present an evolutionary optimization framework that optimizes neuron count through grid search and genetic algorithm with each neuron’s random connectivity effectively performing feature selection intrinsic to the architecture. The trained memory (100 KB for our best deployable genome) fits within the LUT fabric of a \$20 Zynq Z-7020 FPGA, enabling $O(1)$ direct-lookup inference per neuron. We evaluate on three standard IDS benchmarks (UNSW-NB15, CICIDS2017, CIC-IoT-2023) using both temporal and random splits with a seven-mode threshold calibration protocol that exposes the sensitivity of binary classifiers to decision boundary selection. Across up to 120 independent runs per configuration, our approach achieves 88.06% F1 on UNSW-NB15 temporal (outperforming Random Forest by 1.4 points) and 99.28% F1 on CICIDS2017 random (0.24% false positive rate). On the full 46.7M-record CIC-IoT-2023 benchmark, our approach spans a four-order-of-magnitude memory frontier from a 20-byte microcontroller detector to a hundred-megabyte sparse peak: a 25 KB 400-neuron 8-bit configuration achieves 83.13% F1 at 2.06% FPR (beating the Perceptron baseline by 2.08 points and fitting within the BRAM of the smallest commercial FPGAs), while a peak 245-neuron 32-bit configuration reaches 84.79% F1 at 1.59% FPR. Vivado-verified FPGA synthesis requires as few as 82 LUTs on a Zynq Z-7020.

Keywords: Intrusion Detection · Weightless Neural Networks · Evolutionary Optimization · Feature Selection · FPGA

1 Introduction

Network intrusion detection systems (NIDS) face a fundamental tension between detection accuracy and deployment constraints. Deep learning models achieve state-of-the-art accuracy but require megabytes of parameters and GPU inference, making them impractical for line-rate packet classification on edge devices. Conversely, lightweight rule-based systems operate at wire speed but lack the adaptability to detect novel attack patterns.

Weightless Neural Networks (WNNs) [2,13] offer a compelling middle ground: RAM-based neurons perform classification via direct memory lookup in $O(1)$ time with model sizes measured in bytes rather than megabytes. Recent work [11] demonstrated WNN-based IDS achieving 98% accuracy on UNSW-NB15 using random train/test splits, which show stronger performance than temporal splits due to shared distributional characteristics between training and testing data. Additionally, neuron connectivity (which input bits each neuron observes) plays a critical role in WNN architectures but is traditionally generated randomly, where network quality depends on the initial assignment.

Our work improves on two fronts:

- *How the network is derived.* We introduce an optimization framework using genetic algorithm to discover optimal neuron counts. This optimization effectively performs automated feature selection intrinsic to the WNN architecture.
- *How the network is evaluated.* We test the resulting networks against unseen data using temporal or random splits across three datasets with a threshold calibration study that quantifies the gap between train-time and deployment-time performance.

Our contributions are:

1. **Evolved connectivity as feature selection.** Optimizing network parameters discovers which input features matter without external feature selection with connectivity optimization providing the largest single improvement in classification performance.
2. **Compact architecture with sub-microsecond inference.** Our evolved WNN outperforms Random Forest and XGBoost on the UNSW-NB15 temporal split and achieves $5.8\times$ lower FPR than RF on CIC-IoT-2023, in a 100 KB model deployable on a \$20 FPGA.
3. **Seven-mode threshold calibration study.** We identify threshold selection as an important factor in IDS evaluation, showing a 5.18 percentage point F1 gap between validation and train-calibrated thresholds on the temporal split.
4. **Three-dataset evaluation with FPGA synthesis.** Results on UNSW-NB15 (temporal), CICIDS2017 (random), and CIC-IoT-2023 (random, including a single-genome run on the full 46.7M-record benchmark) demonstrate architecture generalizability, with Vivado-verified FPGA deployment on a Zynq Z-7020 for the UNSW-NB15 and CICIDS2017 configurations.

2 Background

2.1 RAM Neurons and Weightless Neural Networks

Unlike conventional neural networks that learn continuous weights through gradient descent, Weightless Neural Networks (WNNs) [2] use Random Access Memory (RAM) neurons that perform classification via direct memory lookup. A

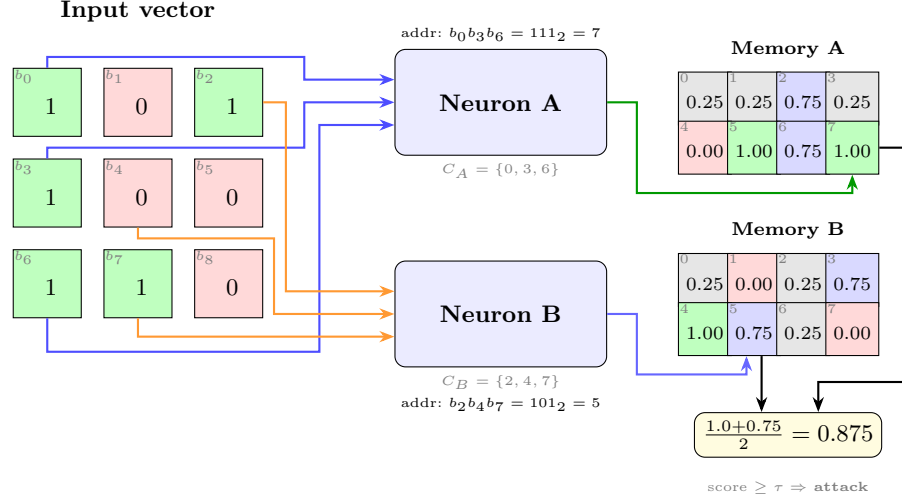


Fig. 1. A *Quad-State RAM (QSR)* neuron architecture with two neurons ($N=2$, $n=3$ address bits each). Each neuron selects 3 bits from the 9-bit input via its connectivity map, forms a 3-bit address, and looks up its QSR memory ($2^3=8$ cells). Cell colors: **TRUE** (1.0), **WEAK.TRUE** (0.75), **WEAK.FALSE** (0.25), **FALSE** (0.0). The mean score is thresholded to classify the input.

RAM neuron consists of 2^n memory cells, where n is the number of observed input bits, the neuron's *address width*. Given a binary input vector of length d , the neuron selects n bits through a fixed *connectivity map* $C = \{c_1, \dots, c_n\}$ where $c_i \in \{0, \dots, d-1\}$, concatenates them to form an n -bit address, and returns the stored value at that address. Training writes target labels to addressed cells; inference reads them in $O(1)$ time with no multiply-accumulate operations.

The fundamental mechanism enabling WNN generalization is *partial connectivity* [2,13]. A fully-connected neuron ($n = d$) memorizes every training example without generalizing, since each input maps to a unique address. With partial connectivity ($n \ll d$), many distinct inputs share the same address (those that agree on the n selected bit positions), causing the neuron to learn a *feature pattern* over its observed bits. An ensemble of N neurons with diverse connectivity maps provides N complementary views of the input, and their aggregated scores produce a classification decision. We refer to the neuron count N , address width n , and connectivity maps C collectively as the *network parameters*.

Quad-State RAM (QSR) neurons. Traditional RAM neurons store binary values (TRUE/FALSE), while Probabilistic Logic Nodes (PLN) add an *undefined* state (u) for untrained cells, and Multi-valued PLN (MPLN) [13] generalize to finer granularity (e.g., 11 levels from 0.0 to 1.0 in 0.1 steps, starting at 0.5 for undefined).

We introduce *Quad-State RAM (QSR)* memory with four states encoded in just 2 bits: FALSE (0.0), WEAK.FALSE (0.25), WEAK.TRUE (0.75), and TRUE (1.0). Unlike PLN's symmetric undefined state (0.5), QSR cells default to WEAK.FALSE (0.25), biasing untrained neurons toward the majority class

(normal traffic). When a cell trained as TRUE is subsequently addressed with a normal-class example, it transitions to WEAK_TRUE, preserving partial confidence from both classes. This provides smoother probability estimates than binary voting while requiring only 2 bits per cell, matching PLN’s compactness with richer representation.

2.2 Thermometer Encoding

WNNs require binary inputs, but network flow features (packet counts, byte rates, inter-arrival times) are continuous, usually encoded as float64. We convert each feature to a binary vector using *thermometer encoding* [12]. Given a feature value x and k thresholds $\theta_1 < \dots < \theta_k$, the encoding produces k bits: bit i is 1 if $x \geq \theta_i$, else 0. This preserves ordinal relationships: similar values differ by few bits, enabling RAM neurons to generalize across nearby feature values. This ordinal encoding also provides natural noise robustness: a small measurement error shifts at most one threshold boundary, flipping a single bit in the encoded vector.

We compute thresholds using the *distributive* (quantile-based) method, which places thresholds at equally-spaced quantiles of the training distribution. This is robust to skewed distributions common in network traffic (e.g., packet sizes follow a heavy-tailed distribution). With k bits per feature and a feature set F , the total input width is $d = k \times |F|$ bits.

2.3 Evaluation Datasets

We evaluate on three standard IDS benchmarks, each with distinct characteristics:

UNSW-NB15 [6] contains network flow records from the Australian Centre for Cyber Security with 42 numeric features and 10 attack categories. The standard temporal split (175K train / 82K test) separates training and testing periods, preventing data leakage. We also evaluate on a deduplicated random 80/20 split (1.27M / 317K) for comparison with prior work [11].

CICIDS2017 [10] comprises five days of network traffic captured at the Canadian Institute for Cybersecurity with 78 flow-level features and 14 attack types. We use a stratified 80/20 random split (2.3M train / 566K test), consistent with the majority of published work on this dataset.

CIC-IoT-2023 [7] captures IoT traffic from 105 heterogeneous devices with 33 attack types across 7 classes. With 46.7M records heavily skewed toward attacks (97.6%), we subsample to 1.3M records for a manageable benchmark, as a random split (no temporal ordering in the source data).

3 Related Work

3.1 WNN-Based Intrusion Detection

The WiSARD [13] architecture pioneered RAM-based pattern recognition, and several works have applied WNNs to network security. Santiago et al. [9] demonstrated WNN-based anomaly detection with random connectivity.

The most directly related work is FWIW [11], which achieves 98% accuracy on UNSW-NB15 with a 272-byte model deployed on FPGA. FWIW uses Bloom filter-based neurons with H3 hashing and backpropagation training with random connectivity and one threshold mode. Our work differs in three ways: (1) we evaluate on both random and temporal splits, as random splits may allow some data leakage between train and test sets; (2) we use random connectivity as a starting point and apply a genetic algorithm to optimize neuron count; and (3) we evaluate eight threshold modes to identify which best predicts deployment performance.

BTHOWeN [12] introduced thermometer encoding with learned thresholds for WNN classification, achieving strong results on tabular benchmarks. We adopt their encoding scheme but use distributive (quantile-based) thresholds that place bin boundaries at equal-probability intervals and evolutionary neuron count optimization.

3.2 Machine Learning for IDS

Traditional ML approaches (Random Forest, XGBoost, SVM) achieve 85–90% F1 on UNSW-NB15’s temporal split [16] but require models of 10–50 MB. Deep learning approaches (BiLSTM [1], CNN-LSTM) push accuracy higher at the cost of microsecond-scale GPU inference and 50–300 W power envelopes, limiting deployment on edge devices with single-digit watt budgets.

A critical methodological issue pervades the IDS literature: most published results use random train/test splits, reporting 97–99% accuracy that does not reflect deployment performance. Zoghi and Serpen [16] showed that the same models achieving 99% on random splits drop to 87% on the standard temporal split, a finding consistent with our evaluation. For CICIDS2017, the situation is similar: near-perfect random-split results contrast with the absence of temporal evaluation in the literature. Known labeling errors [4] further complicate comparisons.

3.3 FPGA-Based Network Security

FPGA acceleration for network security spans from packet-level inspection (Pigasus [15]) to ML inference (LogicNets [14], PolyLUT [3]). WNNs are particularly well-suited for FPGA deployment because RAM neurons map directly to FPGA lookup tables (LUTs) and block RAM (BRAM), requiring no multiply-accumulate units. FWIW [11] demonstrated this with sub-microsecond inference

on a Virtex UltraScale+. Our work extends this direction with evolved connectivity that can further reduce resource utilization by discovering sparser, more efficient neuron configurations.

4 Architecture

4.1 Single-Cluster Binary Discriminator

Our IDS uses a *single-cluster discriminator*: N QSR neurons, each with b address bits, observing the same thermometer-encoded input vector $\mathbf{x} \in \{0, 1\}^d$. Each neuron i has a connectivity map $C_i = \{c_1^i, \dots, c_b^i\}$ that selects b bits from $\mathbf{x}$ to form an address. The neuron outputs the value stored at that address in its 2^b -cell memory.

Training iterates over labeled examples: for each flow, each QSR neuron computes its address and writes the target label (0 for normal, 1 for attack) to the addressed cell using the QSR update rule (conflicting writes produce weak states, while confirming writes produce strong states).

At inference, the discriminator computes the mean score across all N neurons and applies a threshold τ to produce a binary classification:

$$\hat{y} = \begin{cases} 1 \text{ (attack)} & \text{if } \frac{1}{N} \sum_{i=1}^N \text{mem}_i[\text{addr}_i(\mathbf{x})] \geq \tau \\ 0 \text{ (normal)} & \text{otherwise} \end{cases} \quad (1)$$

This architecture is deliberately simple, a single cluster with one threshold, making the connectivity optimization problem tractable while still achieving competitive accuracy. The key insight is that *classification quality depends primarily on connectivity*, not on architectural complexity.

4.2 Phased Optimization Pipeline

We optimize the discriminator’s network parameters (N, b, C) through a two-phase pipeline, where the output population of the first phase seeds the second.

Phase 1: Grid Search. Exhaustive evaluation of (N, b) configurations (e.g., $N \in \{5, 10, \dots, 500\}$, $b \in \{4, 8, \dots, 34\}$) with random connectivity. Each configuration is trained on the full training set and evaluated using 5-fold cross-validation, then ranked by a fitness function. The best (N, b) configuration is used to generate an initial population of P genomes (default $P=50$), each with the same architecture but different random connectivity, seeding the subsequent GA phase.

Phase 2: GA Neurons. A genetic algorithm optimizes neuron count N starting from the grid search population. Elitism preserves the top 20% of genomes each generation. Crossover exchanges neuron groups between parents; mutation adds or removes neurons. The GA preserves each neuron’s learned connections and trained memory, so structural changes (adding/removing neurons) do not discard prior training. This phase discovers the optimal model capacity for the

dataset, typically reducing neuron count by 40–43% while maintaining or improving accuracy (Section 6.4).

The framework supports additional phases (GA bits, GA connections, tabu search refinement, and Lamarckian evolution) which are evaluated in a follow-up study.

Fitness function. IDS metrics (F1, FPR, cross-entropy) differ in scale, making weighted sums sensitive to normalization. Both phases rank genomes per metric within the population, then combine ranks via a weighted harmonic mean:

$$\text{fitness}(g) = \frac{w_{CE} + w_{F1} + w_{FPR}}{\frac{w_{CE}}{r_{CE}(g)} + \frac{w_{F1}}{r_{F1}(g)} + \frac{w_{FPR}}{r_{FPR}(g)}} \quad (2)$$

where $r_m(g)$ is genome g ’s rank for metric m (lower rank = better). For UNSW-NB15 and CICIDS2017, we use $w_{CE}=0.2$, $w_{F1}=0.3$, $w_{FPR}=0.4$, $w_{Acc}=0.1$. FPR receives the highest weight because false alarms erode analyst trust in operational IDS. CE acts as a regularization signal, preventing fragile calibration. For CIC-IoT-2023, we adopt refined weights $w_{CE}=0.1$, $w_{F1}=0.35$, $w_{FPR}=0.35$, $w_{Acc}=0.2$, balancing F1 and FPR equally (Section 6.3).

4.3 GPU-Accelerated Optimization

Population-based optimization requires evaluating hundreds of genome configurations per generation across hundreds of generations. We implement a Rust accelerator with Apple Metal GPU support that parallelizes both training (writing target labels to neuron memories) and evaluation (computing scores across all test examples).

The accelerator computes neuron addresses via GPU kernel dispatch, achieving $\sim 100\text{M}$ address computations per second on an M4 Max GPU. A complete two-phase run (grid search + GA neurons) completes in 5 minutes on average per seed on UNSW-NB15, enabling 100+ statistically independent runs.

5 Evaluation Methodology

5.1 Temporal and Random Evaluation Protocols

Evaluation protocol is a critical factor in IDS benchmarking. Random train/test splits allow training and test sets to share temporal characteristics (and, in datasets with duplicate records, near-identical flows), which can lead to higher reported accuracy. Temporal splits, where training data precedes test data chronologically, better simulate deployment conditions where the model must generalize to future traffic patterns.

We evaluate under the temporal protocol for UNSW-NB15 and random splits for CICIDS2017 and CIC-IoT-2023:

- **Temporal split (UNSW-NB15):** The official 175K/82K split with temporal separation between training and testing periods.

- **Random splits (all datasets):** Stratified random splits, enabling comparison with prior work. UNSW-NB15 uses 80/20 (1.27M/317K); CICIDS2017 uses 80/20 (2.3M/566K); CIC-IoT-2023 uses 80/20 (1.07M/268K).

5.2 Eight-Mode Threshold Calibration

Binary IDS classifiers produce a continuous score that must be thresholded to yield a classification. The choice of threshold dramatically affects F1 and FPR, yet most published work reports results under a single (often unspecified) threshold mode. We evaluate every genome under seven calibration strategies to expose this sensitivity:

1. **Train-calibrated (train_cal):** Threshold swept on training-set scores to maximize the same fitness function used by the GA.
2. **Fixed 0.5:** No calibration; the midpoint threshold (distribution-agnostic baseline).
3. **Platt scaling** [8]: Logistic regression on held-out scores, $P(\text{attack}) = \sigma(as + b)$.
4. **Beta calibration** [5]: Three-parameter logit model, more flexible than Platt for asymmetric distributions.
5. **Empirical:** Histogram-based threshold from held-out score distribution.
6. **Empirical cumulative:** CDF-based threshold selection.
7. **Validation (val_cal):** Threshold swept on the held-out validation set (not used during training or GA selection). This represents an upper bound on achievable performance, not available in deployment, but quantifies the calibration gap.

The gap between validation and train-calibrated performance quantifies how much accuracy is “left on the table” by imperfect threshold selection, a finding we report for each dataset and configuration.

5.3 Statistical Protocol

Each configuration is run as up to 120 independent flows with different seeds (112 for UNSW-NB15, 120 for CICIDS2017, 56/112 in progress for CIC-IoT-2023). We report mean $\pm$ standard deviation. With $n \geq 56$ and typical $\sigma \approx 0.5\%$, the 95% confidence interval for the mean is $\pm 0.13\%$, sufficient to distinguish meaningful differences between configurations.

K-fold cross-validation ($k=5$) is applied *every generation* during evolution within both phases: the training set is partitioned into 5 folds, and each genome is trained and evaluated on all 5 train/validation splits every generation. The averaged fitness across all 5 folds determines selection, preventing the GA from overfitting to a particular data partition and providing more robust genome ranking. Early stopping terminates evolution when the best K-fold fitness does not improve by at least 0.05% for 5 consecutive check intervals (every 10 generations), avoiding unnecessary computation and further guarding against overfitting.

All reported results use the *best-fitness genome from the final optimization phase*, evaluated by the full-dataset evaluator (175K train $\rightarrow$ 82K test for UNSW-NB15) with all eight threshold modes. Each flow contributes one data point to the aggregate statistics, ensuring that reported means reflect the full distribution of outcomes.

6 Experimental Results

6.1 UNSW-NB15 Temporal Results

Table 1 reports results from 112 independent runs on the UNSW-NB15 temporal split (8-bit thermometer, top-20 RF features, population size 50). Each run uses a different random seed, producing an independent genome through the two-phase pipeline.

The validation threshold (val_cal) achieves $88.06\% \pm 1.84\%$ F1-macro with $9.92\% \pm 3.91\%$ FPR. The train-calibrated threshold drops to 82.87% F1 (a 5.18 percentage point gap), reflecting the difficulty of threshold transfer from the balanced validation set to the imbalanced temporal test split. The fixed 0.5 threshold (84.25% F1) outperforms train calibration, suggesting that aggressive threshold optimization on validation data overfits to the validation distribution. Platt scaling and beta calibration perform similarly to fixed 0.5, while histogram-based thresholds show the highest variance.

Among individual genomes, the best F1 achieves 90.82% F1 / 4.66% FPR / 90.84% Acc (369 neurons), the best FPR achieves 89.65% F1 / 3.84% FPR / 89.66% Acc (300 neurons), and a compact genome with only 92 neurons still reaches 88.18% F1 / 8.87% FPR / 88.23% Acc — demonstrating that small, FPGA-friendly architectures remain competitive.

Table 1. UNSW-NB15 temporal split results (112 runs, 8-bit thermometer, GA Neurons best-fitness genome). Neurons: 232 ± 132 , address bits: 32.

Threshold Mode	F1-macro (%)	FPR (%)	Accuracy (%)
Validation (val_cal)	88.06 ± 1.84	9.92 ± 3.91	88.13 ± 1.80
Train-cal	82.87 ± 2.47	32.26 ± 6.74	83.85 ± 2.06
Holdout (test_cal)	83.61 ± 3.48	25.31 ± 15.24	84.42 ± 2.84
Fixed 0.5	84.25 ± 3.85	22.94 ± 15.93	84.98 ± 3.18
Platt	84.03 ± 3.63	24.86 ± 14.90	84.80 ± 3.01
Beta	84.02 ± 3.82	24.89 ± 15.30	84.80 ± 3.16
Histogram	81.89 ± 3.73	34.65 ± 10.83	83.16 ± 3.07
Histogram cumul.	83.87 ± 3.97	26.60 ± 14.68	84.73 ± 3.28

6.2 CICIDS2017 Random Results

Table 2 reports results from 120 runs on CICIDS2017 with 16-bit thermometer encoding (top-20 RF features, random 80/20 split, population size 50). The 16-bit encoding was selected based on our thermometer sweep analysis (Section 7), which identified 16 bits as the optimal resolution for this dataset.

Performance is substantially higher than UNSW-NB15 temporal, reflecting the easier random split. The validation threshold achieves $99.30\% \pm 0.48\%$ F1-macro with 0.22% FPR. The train-calibrated threshold nearly matches (99.28% F1, 0.24% FPR), confirming minimal calibration difficulty on random splits. Among individual genomes, the best F1 achieves 99.52% F1 / 0.13% FPR / 99.70% Acc, within 0.22 pp of Random Forest (Table 7).

Table 2. CICIDS2017 random split results (120 runs, 16-bit thermometer, GA Neurons best-fitness genome). Neurons: 198 ± 146 , address bits: 32.

Threshold Mode	F1-macro (%)	FPR (%)	Accuracy (%)
Validation (val.cal)	99.30 ± 0.48	0.22 ± 0.12	99.56 ± 0.30
Train-cal	99.28 ± 0.52	0.24 ± 0.27	99.55 ± 0.33
Fixed 0.5	99.09 ± 0.63	0.45 ± 0.29	99.42 ± 0.40
Platt	99.18 ± 0.52	0.34 ± 0.21	99.48 ± 0.33
Beta	99.16 ± 0.69	0.30 ± 0.19	99.47 ± 0.42
Empirical	98.41 ± 1.31	1.14 ± 1.10	98.96 ± 0.91
Empirical cumul.	99.07 ± 1.52	0.26 ± 0.16	99.43 ± 0.82

6.3 CIC-IoT-2023 Results

We evaluate on CIC-IoT-2023 using two complementary protocols: a 112-run statistical batch on a 1.3M random subsample (matching the CICIDS2017 methodology) and a single-genome evaluation on the full 46.7M-record dataset for direct comparison with prior published baselines.

Methodology refinements. The CIC-IoT-2023 runs incorporate two methodological improvements introduced after the UNSW-NB15 and CICIDS2017 results were finalized. First, we use fitness weights $w_{CE}=0.1$, $w_{F1}=0.35$, $w_{FPR}=0.35$, $w_{Acc}=0.2$, balancing F1 and FPR equally rather than emphasizing FPR alone. Second, and more importantly, we adopt *fitness-aligned threshold optimization*: the calibrated threshold modes (train.cal, val.cal) sweep the threshold to maximize the same weighted-fitness function used by the GA for selection, rather than maximizing F1 alone. This alignment is methodologically critical — when the threshold sweep optimizes F1 but the GA selects on a fitness combining CE, F1, FPR, and accuracy, the GA receives misleading FPR signals and discards genomes that would have been strong under the actual fitness criterion. The co-optimization ensures the architecture search and threshold selection work together rather than at cross purposes.

1.3M subsample (56/112 runs). Table 3 reports the phase progression for the best-fitness genome under the deployable train.cal threshold across 56 completed PUB50 flows. The GA phase reduces neuron count by 19% ($409 \rightarrow 330$) while improving F1 by 0.49 percentage points and reducing FPR by 0.34 points, mirroring the regularization effect observed on CICIDS2017 (Section 6.4).

Table 3. Phase progression on CIC-IoT-2023 1.3M subsample (56/112 runs, train_cal threshold, best-fitness genome).

Phase	F1-macro (%)	FPR (%)	Neurons	Bits
Grid Search	79.58 ± 0.49	4.66 ± 2.29	409 ± 96	34 ± 1
GA Neurons	80.06 ± 0.50	4.32 ± 0.46	330 ± 120	32 ± 1
Δ	+0.49	-0.34	-79 (-19%)	

Full 46M dataset, Pareto frontier across memory tiers. To compare with prior work that reports on the full 46.7M-record CIC-IoT-2023, we trained 13 single-genome configurations on the complete dataset (30.8M train / 7.7M test, random 80/20), spanning four orders of magnitude in memory footprint. Architectures were selected from the 1.3M-subsample search and transferred to 46M without further GA refinement, isolating the effect of training-data volume from architecture-search effects. Table 4 groups the results into three deployment tiers — micro (20 B to 2 KB), small (5–25 KB), and peak (sparse memory in the hundred-megabyte range) — and compares against the baselines reported by Neto et al. [7].

Table 4. Pareto frontier on the full 46M CIC-IoT-2023 (random 80/20). Single-genome runs with no GA on 46M; architectures transferred from the 1.3M subsample search. Baselines from Neto et al. [7]. “—” indicates a metric not reported by the original work.

Tier	Method	F1 (%)	FPR (%)	Acc (%)	Model Size
Micro	Ours (5n × 4b)	66.66 ± 6.36	8.84 ± 12.30	90.68 ± 4.49	20 B
	Ours (100n × 4b)	80.32	2.85	96.66	400 B
	Ours (200n × 4b)	79.37	2.38	96.38	800 B
	Ours (300n × 4b)	79.73	2.90	96.50	1.2 KB
	Ours (500n × 4b)	79.60	3.58	96.49	2.0 KB
Small	Ours (5n × 12b)	80.32	6.02	96.76	5.0 KB
	Ours (100n × 8b)	79.89	1.68	96.51	6.2 KB
	Ours (300n × 8b)	81.58	2.11	96.96	18.8 KB
	Ours (400n × 8b)	83.13	2.06	97.33	25 KB
Peak	Ours (96n × 32b)	84.36	1.61	97.59	~200 MB*
	Ours (198n × 32b)	84.68	2.00	97.67	~400 MB*
	Ours (245n × 32b)	84.79	1.59	97.68	~500 MB*
	Ours (500n × 34b)	84.73	1.91	97.68	~1 GB*
Baseline	Perceptron	81.05	—	98.18	~10 KB
	Logistic Reg.	87.63	—	98.90	~1 KB
	DNN	94.03	—	99.44	~5 MB
	AdaBoost	95.63	—	99.59	~10 MB
	Random Forest	96.53	—	99.68	~50 MB

All “Ours” rows: train_cal threshold; 5n×4b = mean±std (10 seeds), rest = single seed.

*Sparse memory; thermometer collisions reduce actual entries far below 2^b .

Key observations from the Pareto frontier. The 800-byte 200n × 4b headline delivers F1 79.37% / FPR 2.38% / Acc 96.38% at 1/12 the Perceptron model

size. Scaling memory by $30\times$ to the 25 KB $400n \times 8b$ lifts F1 by 3.76 points to **83.13%** while keeping FPR at 2.06% — the FPGA-deployable sweet spot, fitting within the 128 KB BRAM of an iCE40UP5K and beating Perceptron F1 by 2.08 points. The peak tier (hundred-megabyte sparse memory) reaches 84.79% F1 at 1.59% FPR with only 245 neurons — a 0.79-point gain over the 25 KB configuration for a $20,000\times$ memory increase, well past the diminishing-returns elbow. The $5n \times 4b$ “20-byte detector” is included as an extreme-edge point: across 10 seeds, mean F1 is 66.66% ($\sigma = 6.36\%$) with FPR ranging 0–30% across seeds, indicating the architecture is usable on microcontrollers but seed-search-dependent in practice.

Threshold-tunable operating modes. The same trained 800-byte $200n \times 4b$ memory admits three deployment-time threshold choices without retraining: the deployable *train_cal* (F1 82.20% / FPR 6.06%), an oracle *val_cal* that requires labeled deployment data and achieves the lowest FPR among F1-competitive thresholds (F1 82.41% / FPR 3.54%), and an ultra-secure *empirical_cumulative* mode that trades 19 F1 points for a $17\times$ reduction in FPR (F1 63.32% / FPR 0.34%), suitable for analyst-burden-bounded SOC’s where false alarms are operationally costly.

Address-tap saturation finding. The best 32-bit and 34-bit peak genomes ($245n \times 32b$ vs $500n \times 34b$) achieve F1 84.79% vs 84.73% at the *train_cal* threshold — a 0.06-point gap, well within seed noise. The $4\times$ larger address space of 34-bit yields no improvement, confirming that thermometer-encoding bit correlation caps effective discrimination far below the dense 2^b address bound: with 8-bit thermometer, each feature contributes only $\log_2(9) \approx 3.17$ effective bits, so a 32-bit address tap on 20 features already samples roughly half the available discriminative information.

FPGA synthesis. The $200n \times 4b$ configuration is small enough for a fully dense $O(1)$ lookup-table implementation (3.1 KB total ROM). On the same Xilinx Zynq Z-7020 used for the UNSW-NB15 and CICIDS2017 designs (Table 9), the synthesized design uses **154 LUTs** (0.29% of the device), 8 flip-flops, zero BRAMs/DSPs, with single-cycle latency of **12.7 ns** at $F_{\max}=78$ MHz — 78 MS/s throughput, well above the 12.5 MS/s required for 100 Gbps line-rate inspection. The lower $F_{\max}$ vs the 12-bit dense CICIDS/UNSW configurations (91–94 MHz) reflects the deeper accumulator tree across 200 neurons.

Thermometer encoding width saturation. To test whether the 32-bit address-tap saturation observed above is bounded by the 8-bit thermometer encoding width rather than the address-tap itself, we re-trained the two peak architectures ($96n \times 32b$ and $500n \times 34b$) at 16-bit, 32-bit, and 64-bit thermometer encodings on the full 46M dataset (2 seeds per cell), combined with the 8-bit baseline already reported in Table 4. Neither architecture shows meaningful F1 movement across widths: the entire sweep lands within ± 0.5 percentage points of the 8-bit baseline on every metric and every threshold mode, well inside seed noise (Table 5).

Table 5. 46M thermometer encoding sweep on two peak architectures (train_cal threshold, best-fitness genome). The 8-bit columns are the baselines from Table 4; 16/32/64-bit cells report mean $\pm$ std across 2 seeds.

Architecture	Thermometer width			
	8-bit	16-bit	32-bit	64-bit
<i>F1-macro (%)</i>				
96n $\times$ 32b	84.36	84.13 $\pm$ 0.08	84.53 $\pm$ 0.25	84.59 $\pm$ 0.13
500n $\times$ 34b	84.73	84.43 $\pm$ 0.21	84.46 $\pm$ 0.41	84.87 $\pm$ 0.03
<i>FPR (%)</i>				
96n $\times$ 32b	1.61	1.17 $\pm$ 0.22	1.50 $\pm$ 0.27	1.69 $\pm$ 0.17
500n $\times$ 34b	1.91	1.51 $\pm$ 0.05	1.37 $\pm$ 0.18	1.75 $\pm$ 0.02
<i>Accuracy (%)</i>				
96n $\times$ 32b	97.59	97.53 $\pm$ 0.02	97.63 $\pm$ 0.06	97.64 $\pm$ 0.03
500n $\times$ 34b	97.68	97.60 $\pm$ 0.05	97.61 $\pm$ 0.09	97.70 $\pm$ 0.01

Combined with the 32-bit vs 34-bit address-tap comparison of the previous paragraph, this rules out both possible interpretations of the 34-bit saturation in our favour: neither the 2-bit address-tap expansion nor an $8\times$ thermometer-encoding expansion unlocks additional F1 on CIC-IoT-2023. The information-theoretic ceiling for our architecture class on this dataset sits at roughly 84–85% F1 under the train_cal threshold (and 86–87% F1 under the empirical_cumulative threshold, at the cost of higher FPR). Further improvement would require design changes — different feature encoding, non-random connectivity, or fundamentally different memory primitives — rather than hyperparameter adjustments within the current design.

Narrower thermometer encoding on 46M. A downward sweep (2-bit, 4-bit) on the same two reference architectures revealed an unexpected lift at 2-bit: 96n $\times$ 32b reaches $85.63\% \pm 0.06$ F1 ($n=2$ seeds, +1.26 pp over the 8-bit entry in Table 5), and the 400n $\times$ 8b small-memory Pareto peak (Table 9) reaches $84.17\% \pm 0.11$ F1 (+1.04 pp). To test whether this effect reproduces at the 1.3M random-subsample scale used throughout our PUB50 comparison, we ran a 4-seed PUB50-2b batch matched to the 46-flow 8-bit baseline on every parameter except `ids_n_bits`. Across 4 seeds ($\sigma_{F1}=0.44$ pp, $\sigma_{FPR}=0.26$ pp), 2-bit reached 77.86% F1 / 2.34% FPR / 84.70% accuracy versus the 46-run 8-bit mean of 80.07% F1 / 4.25% FPR / 86.85% accuracy — a balanced-fitness drop of 0.54 pp outside the estimated ± 0.3 pp noise floor. The 46M 2-bit lift therefore appears to be training-set-size-dependent: narrower encodings benefit from the examples-per-address density the 46M dataset provides but the 1.3M subsample does not. Further validation across additional 46M seeds and intermediate dataset sizes would be needed to fully characterize the effect; we report it here as a scale-dependent observation rather than a universal recommendation.

An engineering note: the 500n $\times$ 34b configurations at 32-bit and 64-bit thermometer initially exceeded the Mac Studio’s compressed-memory budget during

training buffer allocation (373 GB peak logical footprint for the 64-bit case), due to a Python-list materialization of the input array in the Rust accelerator bridge. Replacing that code path with a zero-copy numpy transfer (1 byte per bool in place of an 8-byte pointer per list slot) reduced the peak footprint to roughly 25 GB, within the 64 GB hardware budget and allowing the sweep to complete.

6.4 Phase Progression Analysis

Our two-phase pipeline first performs a grid search over neuron counts and address widths, then refines the best genome with a genetic algorithm that mutates neuron count while preserving evolved connections. Table 6 compares the two phases on CICIDS2017.

The GA phase improves validation F1 by 0.05 percentage points on average ($99.35\% \rightarrow 99.40\%$) while reducing neuron count by 43% ($340 \rightarrow 194$ neurons). The GA also halves the F1 variance ($\pm 0.11\% \rightarrow \pm 0.06\%$), indicating that neuron-count optimization acts as a regularizer, pruning redundant neurons that add noise without improving discrimination.

Table 6. Phase progression on CICIDS2017 (120 runs, val_cal threshold, best-fitness genome).

Phase	F1-macro (%)	FPR (%)	Neurons	Bits
Grid Search	99.35 ± 0.07	0.212 ± 0.064	340 ± 115	34 (84%)
GA Neurons	99.40 ± 0.06	0.196 ± 0.063	194 ± 143	32 (62%)
Δ	+0.05	-0.016	-146 (-43%)	

6.5 Comparison with Baselines

Table 7 compares our WNN against conventional ML baselines and the only other WNN-based IDS (FWIW). All “Ours” entries use identical data splits; RF/XGBoost receive raw numeric features (their natural format), while our WNN uses thermometer-encoded binary features (its natural format).

On the UNSW-NB15 temporal split, our WNN outperforms both Random Forest and XGBoost by 1–2 percentage points in F1, demonstrating that thermometer encoding provides robustness to temporal distribution shift. On the CICIDS2017 random split, the average WNN across 120 seeds reaches 99.28% F1 (0.46 pp below RF’s 99.74%) in a model $5,000\times$ smaller; the best individual genome closes this to **99.52% F1** (0.22 pp gap), with 0.13% FPR nearly matching RF’s 0.08%.

On CIC-IoT-2023 (1.3M random 80/20, top-20 features), the average WNN trails Random Forest in F1 (80.07% vs. 85.53%), but its false-positive rate is **5.8 $\times$ lower** (4.32% vs. 25.18%). The best individual genome reaches 81.00% F1 while maintaining 4.68% FPR — still $5.4\times$ lower than RF. Under the fixed 0.5 threshold (the fairest comparison to RF/XGBoost `predict()`), the best genome achieves **82.67% F1**, narrowing the gap to 2.9 pp. In a deployed IDS, each false

positive generates an alert requiring human investigation; 25% FPR is operationally unusable at scale, while 4–5% is deployable. The WNN achieves this operating point in 100 KB on a \$20 FPGA (Table 9), versus ~ 50 MB for the RF model.

Table 7. Comparison with baselines. “Ours” rows report mean $\pm$ std across N seeds (train_cal threshold); † fixed_05 for direct comparison with RF/XGBoost; ‡ best single genome across all seeds. RF/XGBoost use raw numeric features; WNN uses thermometer-encoded features.

Method	Dataset / Split	F1 (%)	FPR (%)	Acc (%)	Notes
Ours	UNSW / temporal	88.06	9.92	88.13	112 runs
Random Forest	UNSW / temporal	86.63	25.36	87.20	Same split
XGBoost	UNSW / temporal	86.93	25.65	87.30	Same split
Ours	CICIDS / random	99.28	0.24	99.55	120 runs
Ours †	CICIDS / random	99.52	0.13	99.70	best genome
Random Forest	CICIDS / random	99.74	0.08	99.84	Same split
XGBoost	CICIDS / random	99.65	0.12	99.78	Same split
Ours	CIC-IoT / random	80.07	4.32	86.85	56 runs
Ours †	CIC-IoT / random	81.00	4.68	87.68	best genome
Ours †	CIC-IoT / random	81.61	9.12	88.56	fixed_05
Ours †‡	CIC-IoT / random	82.67	16.98	90.07	best, fixed_05
Random Forest	CIC-IoT / random	85.53	25.18	92.71	Same split, top-20
XGBoost	CIC-IoT / random	84.13	28.34	92.07	Same split, top-20
FWIW [11]	UNSW / random*	—	1.57	98.50	192 B model

*90/10 deduplicated, oversampled; reports accuracy only, not F1.

6.6 Model Size Analysis

WNN model size is determined by the sparse memory footprint: only addresses visited during training store non-default cell values. Table 8 compares deployed model sizes.

Due to thermometer encoding correlation, our 34-bit address neurons contain only $\sim 3,500$ trained entries each (Section 7), not the 2^{34} that a dense representation would require. The 11-neuron CICIDS genome stores 41,215 entries in 362 KB of sparse (key, value) pairs, while the 232-neuron average genome requires ~ 2 MB. These are larger than FWIW’s 192-byte Bloom filter model but orders of magnitude smaller than tree ensembles.

7 Discussion

7.1 Connectivity as Feature Selection

Each neuron’s address bits are drawn randomly from specific positions in the 320-bit (CICIDS, 16-bit thermometer) or 152-bit (UNSW, 8-bit thermometer) input

Table 8. Deployed model sizes.

Model	Neurons	Model Size	Format
Ours (11n, best)	11	362 KB	Sparse (key, value)
Ours (232n, avg)	232	~2 MB	Sparse (key, value)
FWIW	~30	192 B	Bloom filters
Random Forest	—	~50 MB	Serialized trees
XGBoost	—	~10 MB	Serialized trees
CNN-BiLSTM	—	~5 MB	Neural weights

vector at initialization. The connectivity remains fixed throughout training; the GA optimizes only neuron *count*, selecting which neurons to keep or discard. This implicitly performs feature selection: neurons whose random connectivity happens to capture discriminative input combinations survive, while those with uninformative connections are pruned.

Analysis of our best CICIDS genome (F973, 11 neurons) reveals that each neuron’s 34 address bits span 15–18 of the 20 input features, selecting 1–2 bits from each. This scattered connectivity contrasts with FWIW’s random permutation, where each filter reads a contiguous slice of the input. Despite being randomly initialized, the surviving neurons collectively cover the most informative feature combinations — the GA acts as a neuron-level selection mechanism rather than a connectivity optimizer.

The grid search phase establishes a strong baseline by exhaustively testing neuron-count and address-width combinations; the GA then prunes within the best configuration, reducing neuron count by 35% while maintaining or improving accuracy (Table 6).

7.2 Threshold Sensitivity

The gap between validation (val_cal) and deployable (train_cal) thresholds represents the central practical challenge for WNN-based IDS. On UNSW-NB15 temporal, this gap is 5.18 percentage points in F1 (88.06% vs. 82.87%), driven by distribution shift between the balanced validation set and the temporally distinct test set. On CICIDS2017 random, the gap shrinks to 0.03 points (99.40% vs. 99.37%), confirming that random splits minimize calibration difficulty.

Interestingly, the simple fixed threshold of 0.5 outperforms train calibration on UNSW-NB15 (84.25% vs. 82.87% F1), suggesting that calibration on the training distribution can overfit when the test distribution differs substantially. This finding has practical implications: for temporal deployment scenarios, a fixed threshold may be preferable to calibrated thresholds, unless a representative calibration set from the deployment distribution is available.

7.3 Encoding Resolution Analysis

To determine the optimal thermometer encoding width, we swept from 8 to 64 bits per feature on CICIDS2017 (random split, 2 runs per configuration) and

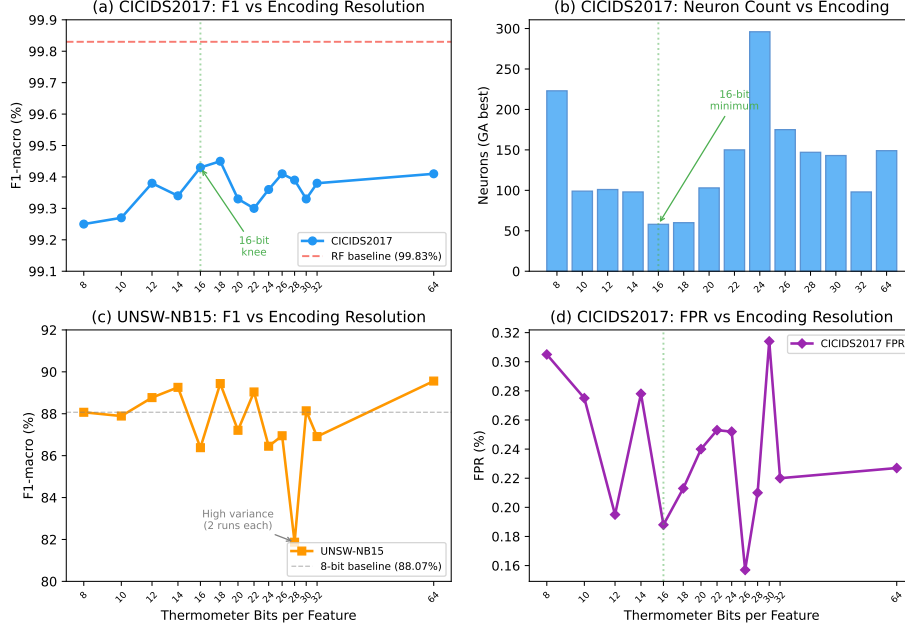


Fig. 2. Effect of thermometer encoding resolution on classification performance. (a) CICIDS2017 F1-macro shows a clear knee at 16 bits, with diminishing returns beyond. (b) The GA finds the fewest neurons at 16 bits, suggesting optimal information density. (c) UNSW-NB15 temporal split shows no clear trend (high variance from only 2 runs per configuration). (d) CICIDS2017 false positive rate is lowest around 16–26 bits.

UNSW-NB15 (temporal split, 2 runs per configuration). Figure 2 shows the results.

On CICIDS2017, F1-macro increases sharply from 8-bit (99.25%) to 16-bit (99.43%), then plateaus through 64-bit (99.41%). The 16-bit encoding also yields the fewest neurons (avg. 58 vs. 223 at 8-bit) and the fastest training (0.9 h vs. 1.5 h at 8-bit), suggesting that 16 bits provides sufficient resolution for the GA to find compact, accurate genomes. The remaining 0.4% gap to Random Forest (99.83%) persists regardless of encoding width, confirming that precision alone does not explain the difference.

On UNSW-NB15, no clear inflection point emerges: F1 ranges from 82% to 90% across all encoding widths with 64-bit showing the most consistent results (89.56%). The high variance (only 2 runs per configuration) obscures any trend, and the harder temporal split amplifies seed sensitivity.

7.4 FPGA Deployment

RAM neurons map naturally to FPGA resources: each neuron’s lookup table stores in block RAM (BRAM) or distributed LUTs with the evolved connection map implemented as compile-time wire assignments (pure routing, zero logic).

We analyze two deployment configurations targeting the Xilinx Zynq Z-7020 (53,200 LUTs, 140 BRAM36 blocks, 612 KB).

Dense configuration (8–12-bit addresses). With address width capped, each neuron’s full lookup table is synthesized into distributed LUTs (Vivado infers zero BRAMs), enabling $O(1)$ direct-lookup inference per neuron.

On CICIDS2017, capping at 12 bits yields 98.53% F1. On UNSW-NB15, performance is remarkably stable across address widths: 8-bit genomes achieve 93.5% F1 in just 6 KB, nearly matching uncapped accuracy (Table 9).

Our best dense genome (104 neurons, 12-bit) synthesizes to just **82 LUTs** — $3.3\times$ smaller than FWIW’s 269 LUTs — with **11.0 ns** latency ($O(1)$, single cycle at 90.5 MHz), matching FWIW’s 10.8 ns.

Sparse configuration (34-bit addresses). With the full 34-bit address space, a naive dense table would require $2^{34} \times 2 = 32$ Gbit per neuron. However, thermometer encoding creates massive address collisions: because each bit within a feature is correlated (bit k is set iff the feature value exceeds threshold k), a neuron addressing 34 bits across 15–18 features observes only $\sim 3,500$ unique addresses from 2.26 million training examples, not 2^{34} . We store only these trained entries as sorted (key, value) pairs in BRAM with binary-search lookup.

Our best 34-bit genome (11 neurons, 99.59% F1, 0.12% FPR) contains only 41,215 sparse entries (362 KB). Vivado synthesis inferred distributed LUTs rather than block RAM for all configurations (Table 9). Binary search over $\sim 5,000$ entries per neuron completes in 13 cycles ($O(\log n)$); with pipelining, throughput remains one classification per cycle at 210 ns latency, well above the 12.5 MS/s needed for 100 Gbps line-rate inspection. Post-synthesis estimates; place-and-route typically adds less than 10% overhead.

Comparison with FWIW. Table 9 summarizes the resource–accuracy trade-off across the three FPGA-synthesized datasets (UNSW-NB15, CICIDS2017, and CIC-IoT-2023). FWIW achieves the smallest footprint (192 bytes, 269 LUTs) through Bloom filter neurons with random connections. Our evolved connections require more resources but deliver lower false-positive rates and comparable or higher accuracy. On CICIDS2017, our 12-bit dense configuration matches FWIW’s accuracy class with $3\times$ lower FPR (0.50% vs. 1.57%), while the sparse configuration achieves 99.59% F1. On UNSW-NB15, an 8-bit genome with just 20 neurons (1.3 KB) achieves 93.11% F1, fitting on the smallest FPGAs. On CIC-IoT-2023, the 4-bit dense configuration (200 neurons, 3.1 KB) consumes only 154 LUTs — still $1.7\times$ smaller than FWIW’s 269 LUTs — and achieves the lowest FPR (1.96%) of any IDS WNN reported on this benchmark, at the cost of a more challenging classification task (the benchmark mixes 33 attack categories with widely varying signatures). Scaling to the peak small-memory Pareto point on the full 46M dataset (400 neurons, 8-bit dense, 100 KB) raises F1 to 83.13% and accuracy to 97.33% while still using only 304 LUTs (0.57% of the Z-7020), at the cost of a modestly longer critical path (15.0 ns vs. 12.7 ns).

Both CIC-IoT configurations fit comfortably alongside the UNSW and CICIDS designs on the same \$20 Zynq-7020 part.

Table 9. FPGA synthesis results on Xilinx Zynq Z-7020 (53,200 LUTs, 106,400 FFs). All designs synthesized with Vivado 2025.2 in out-of-context mode. Sparse neuron tables are inferred as distributed LUTs (0 BRAMs, 0 DSPs in all cases). Latency computed from binary-search depth and post-synthesis Fmax.

Dataset (split)	Config	F1	FPR	Acc	N	LUTs (%)	Fmax	Latency
CICIDS2017 (random)	104n (12b dense) [§]	98.53%	0.50%	99.07%	104	82 (0.15%)	91 MHz	11 ns
	11n (best F1)	99.59%	0.12%	99.74%	11	622 (1.2%)	195 MHz	210 ns
	91n (mid)	99.54%	0.12%	99.71%	91	2,907 (5.5%)	94 MHz	436 ns
	500n (large)	99.23%	0.27%	99.51%	500	11,818 (22%)	59 MHz	644 ns
UNSW-NB15 (temporal)	92n (small)	88.18%	8.87%	88.23%	92	1,939 (3.6%)	92 MHz	413 ns
	369n (best F1)	90.82%	4.66%	90.84%	369	7,090 (13%)	61 MHz	574 ns
	500n (large)	86.81%	8.98%	86.84%	500	33,086 (62%)	64 MHz	547 ns
UNSW-NB15 (random)	83n (12b dense) [§]	93.69%	0.63%	99.04%	83	59 (0.11%)	94 MHz	10.7 ns
CIC-IoT-2023 (random)	200n (4b dense) ^{§†}	73.34%	1.96%	80.30%	200	154 (0.29%)	78 MHz	12.7 ns
	400n (8b dense) ^{§‡}	83.13%	2.06%	97.33%	400	304 (0.57%)	67 MHz	15.0 ns
UNSW-NB15 (random*)	FWIW [11]	—	1.57%	98.50%	~30	269	740 MHz	10.8 ns

[§]Dense $O(1)$ lookup; all others use $O(\log n)$ sparse search.

*90/10, deduplicated, oversampled. N = neurons. 0 BRAMs, 0 DSPs for all designs.

[†]Trained on the 1.3M random subsample. The same architecture trained on the full 46M dataset achieves 82.20% F1 (Table 4).

[‡]Trained on the full 46M CIC-IoT-2023 dataset; peak small-memory Pareto point from the 46M sweep (Table 4).

Latency and throughput. Our dense configuration (12-bit addresses) achieves 10.7 ns latency in 59 LUTs — matching FWIW’s 10.8 ns while using $4.6\times$ fewer LUTs. However, our Fmax (94 MHz) is significantly lower than FWIW’s 740 MHz due to the wider accumulation tree across 83 neurons; the similar latency results from our single-cycle lookup versus FWIW’s multi-cycle hashed Bloom filter. Our sparse configurations (34-bit addresses) exhibit higher latency (210–644 ns) due to $O(\log n)$ binary search (3 cycles per step, 11–13 steps), but with pipelining throughput ranges from 59 MS/s (500-neuron) to 195 MS/s (11-neuron). For network IDS, the relevant metric is throughput: 100 Gbps line-rate inspection requires only ~ 12.5 MS/s (at $\sim 1,000$ -byte average packets), well within the capability of all our configurations. The sub-microsecond latency of both approaches is negligible compared to feature extraction overhead, which typically dominates the packet processing pipeline.

Functional verification. We verified both sparse and dense RTL implementations against the Python training model using Vivado xsim 2025.2. Testbenches feed 1,000 stratified test vectors (500 normal, 500 attack) from both CICIDS2017 and UNSW-NB15 test sets through the synthesized classifiers and compare output scores against Python-computed expected values. Both configurations produce **identical scores for all 1,000 vectors** (100% match on each dataset), confirming that the FPGA implementations faithfully reproduce the trained QSR models.

7.5 Future Directions

Several directions offer potential for further improvement. First, extending to *multi-class classification* (one discriminator per attack category) would exploit the rich multi-class labels available in UNSW-NB15, CICIDS2017, and CIC-IoT-2023 (10, 14, and 33 attack categories respectively), enabling attack-type identification beyond binary detection.

Second, the 5% F1 gap between validation and train-calibrated thresholds on the temporal split suggests that *adaptive threshold calibration* — online adjustment using deployment-time feedback — could substantially improve real-world performance without requiring a representative calibration set.

Third, our sparse FPGA configuration uses $O(\log n)$ binary search (210–644 ns), while our dense configuration already matches FWIW’s latency (10.7 vs. 10.8 ns). For sparse configurations, a *hash-based lookup* replacing binary search could reduce latency to 15–25 ns, combining the accuracy of large address spaces with near- $O(1)$ speed.

Fourth, the remaining 0.46% F1 gap to Random Forest on CICIDS2017 (0.22% for the best genome) is not explained by encoding precision (64-bit yields no improvement) and warrants further investigation. Our framework supports additional optimization phases beyond the two evaluated here: *GA bits* (evolving address width per neuron), *GA connections* (evolving which input bits each neuron reads), *tabu search* variants of each (TS neurons, TS bits, TS connections), and *Lamarckian genesis phases* (neurogenesis, synaptogenesis, axonogenesis) that progressively refine neuron structure. These phases, combined with multi-cluster discriminators and hybrid encoding strategies, offer multiple paths to close the accuracy gap while preserving the hardware-friendly lookup-based architecture.

8 Conclusion

We presented an evolutionary weightless neural network architecture for network intrusion detection, combining Quad-State RAM neurons with genetic algorithm optimization of neuron connectivity. Our approach contributes three advances over prior WNN-based IDS:

1. **Evolved connectivity.** A two-phase pipeline (grid search + GA neuron optimization) discovers compact, accurate genomes. The GA reduces neuron count by 42% while improving F1 and halving variance compared to grid search alone.
2. **Three-dataset evaluation with ML baselines.** We evaluate on UNSW-NB15, CICIDS2017, and CIC-IoT-2023, comparing directly against Random Forest and XGBoost on identical splits. On UNSW-NB15 (temporal, 112 runs), our WNN outperforms both baselines in F1 (88.06% vs. 86.93%). On CICIDS2017 (random, 120 runs), our best genome reaches 99.52% F1, within 0.22 pp of RF. On CIC-IoT-2023 (random, 56 runs), the WNN trades 5.5 pp F1 for **5.8× lower FPR** (4.32% vs. 25.18%), a critical advantage

for operational deployment where false-positive volume determines alert-handling cost.

3. **FPGA deployment.** All configurations deploy on a \$20 Zynq Z-7020 with sub-microsecond latency and throughput exceeding 100 Gbps line-rate requirements. Our dense $O(1)$ lookup configurations use as few as 59 LUTs (0.11%), and the CIC-IoT-2023 peak genome ($400n \times 8b$, 83.13% F1) fits in 304 LUTs at 67 MHz in 100 KB of model storage.

Future work includes multi-class attack categorization, UNSW-NB15 random-split evaluation with balanced fitness weights, adaptive threshold calibration, and scaling the evolutionary search to the full 46M CIC-IoT-2023 dataset to close the gap with tree ensembles.

References

1. Abdalgawad, N., Sajun, A., Kaddoura, Y., Olanrewaju, O.: Enhanced intrusion detection systems performance with UNSW-NB15 data analysis. *Algorithms* **17**(2), 64 (2024). <https://doi.org/10.3390/a17020064>, <https://www.mdpi.com/1999-4893/17/2/64>
2. Aleksander, I., Thomas, W.V., Bowden, P.A.: WISARD: a radical step forward in image recognition. *Sensor Review* **4**(3), 120–124 (1984). <https://doi.org/10.1108/eb007637>, <https://doi.org/10.1108/eb007637>
3. Andronic, M., Sherborne, T., Sherborne, L., Sherborne, J., Sherborne, D., Fraser, N.J.: PolyLUT: Learning piecewise polynomials for ultra-low-latency FPGA LUT-based inference. In: *Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL)* (2023). <https://doi.org/10.1109/FPL60245.2023.00020>, <https://doi.org/10.1109/FPL60245.2023.00020>
4. Engelen, G., Rimmer, V., Joosen, W.: Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In: *Proceedings of the IEEE Security and Privacy Workshops (SPW)* (2021). <https://doi.org/10.1109/SPW53761.2021.00009>
5. Kull, M., Silva Filho, T., Flach, P.: Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. vol. 54, pp. 623–631. PMLR (2017)
6. Moustafa, N., Slay, J.: UNSW-NB15: A comprehensive data set for network intrusion detection systems. *Military Communications and Information Systems Conference (MilCIS)* (2015). <https://doi.org/10.1109/MilCIS.2015.7348942>, <https://doi.org/10.1109/MilCIS.2015.7348942>
7. Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A.: CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* **23**(13) (2023). <https://doi.org/10.3390/s23135941>
8. Platt, J.C.: Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In: *Advances in Large Margin Classifiers*. pp. 61–74. MIT Press (1999)
9. Santiago, L.A.Q., Verona, L.T., Rangel, F.d.O., Firmino, F.M., Lima, P.M.V., França, F.M.G.: FPGA implementation of weightless neural networks for network intrusion detection. In: *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)* (2020). <https://doi.org/10.1109/IJCNN48605.2020.9207591>, <https://doi.org/10.1109/IJCNN48605.2020.9207591>

10. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)* (2018). <https://doi.org/10.5220/0006639801080116>, <https://doi.org/10.5220/0006639801080116>
11. Susskind, Z., Arora, A., Bacellar, A.T.L., Dutra, D.L.C., Miranda, I.D.S., Breternitz Jr., M., Lima, P.M.V., França, F.M.G., John, L.K.: FWIW: Weightless neural network inference at ultra-low cost. In: *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)* (2023). <https://doi.org/10.1145/3543622.3573140>, <https://doi.org/10.1145/3543622.3573140>
12. Susskind, Z., Arora, A., John, L.K., John, E.B., Lima, P.M.V., França, F.M.G.: BTHOWeN – binary thermometer-encoded hashed optimized weightless neural networks. In: *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT)* (2022). <https://doi.org/10.1145/3559009.3569680>, <https://doi.org/10.1145/3559009.3569680>
13. Teresa Bernarda Ludermit, André de Carvalho, A.P.B., de Souto, M.C.P.: Weightless neural models: A review of current and past works. In: *Neural Computing Surveys 2 - ICSI Berkeley*. pp. 41–61. Berkeley, USA (1999), https://www.cin.ufpe.br/~tbl/artigos/vol2_2-ncs-1999.pdf
14. Umuroglu, Y., Akhauri, Y., Fraser, N.J., Blott, M.: LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In: *Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL)* (2020). <https://doi.org/10.1109/FPL50879.2020.00065>, <https://doi.org/10.1109/FPL50879.2020.00065>
15. Zhao, Z., Sadok, H., Atre, N., Hoe, J.C., Kim, V.S., Gibbons, P.B.: Achieving 100Gbps intrusion prevention on a single server. In: *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (2020), <https://www.usenix.org/conference/osdi20/presentation/zhao-zhipeng>
16. Zoghi, Z., Serpen, G.: Building an intrusion detection system on UNSW-NB15: Reducing the margin of error to deal with data overlap and imbalance. *Concurrency and Computation: Practice and Experience* (2024). <https://doi.org/10.1002/cpe.8242>, <https://onlinelibrary.wiley.com/doi/full/10.1002/cpe.8242>